Implementation guide: This document explains how to use the current Deviation Dash engine as a teacher model for forecasting, mimic models, and hybrid systems.

Focus: what to learn, what to keep hard-coded, how to build training tables, and how to evaluate student models safely.

How To Use Deviation Dash To Train Other Models

Purpose

This guide explains how to use the current Deviation Dash engine as a **teacher model** for other machine-learning models.

The key idea is:

- the current app already contains a large amount of business knowledge
- that knowledge can be turned into labels, features, and guardrails for new models
- not every part of the current engine should be learned the same way

The strongest design is usually a **hybrid**:

- learn demand where learning helps
- keep hard business rules as explicit policy constraints
- use the current model as the source of training targets and explanations

What The Current Model Already Knows

The current Deviation Dash engine is not just a simple average calculator. It already combines:

- status-based ignore rules
- branch vs DC behavior
- supplier frequency and lead time
- service level by ABC class
- seasonality
- year-over-year reference-window logic
- intermittent-demand protection
- low-cost deviation handling
- supplier minimum amount floors

- manual overrides
- branch-zero-max and DC-zero-max alerts

That means it is already a valuable **teacher**.

The Best Ways To Use This Model

There are four realistic ways to use the current engine to train other models.

1. Train A Demand Model, Then Keep The Rules

This is the best first path.

How it works:

- Train a model to predict near-term demand.
- Feed that demand into the existing rules engine.
- Let the existing engine still decide min/max, override behavior, regional behavior, supplier minimums, and alerts.

Why this is best:

- demand is the part most worth learning
- policy is the part least safe to leave to a black box
- it is easier to validate and explain

~~Good model types:~~

- ~~AutoGluon TimeSeries~~
- ~~XGBoost / LightGBM on engineered time-series features~~
- ~~TemporalFusionTransformer or another sequence model if the team wants more complexity~~

Best prediction targets:

- next 1 month demand
- next 2 to 3 months demand
- expected demand over the current protection window

2. Train A Rule-Mimic Model

This means training a model to reproduce the current engine's outputs directly.

Typical targets:

- `recommended_min_amount`
- `recommended_new_max`
- `branch_zero_max_recommendation_alert`
- `dc_pooling_zero_max_alert`
- `regional_zero_max_branch_pool_applied`

Why this can help:

- gives the team a faster surrogate model
- useful for model comparison and distillation
- useful if the team wants to reproduce the current engine in another stack

Main caution:

- this teaches the model policy-adjusted outputs, not pure demand
- if the inputs miss important rule context, the mimic model will look unstable or contradictory

3. Train A Human-Decision Model

This only becomes strong if you also have planner actions over time.

Targets:

- whether a buyer accepted the recommendation
- how much they changed min/max
- whether an override was added later
- downstream outcomes such as stockout, fill rate, turns, or excess

This is the best path if you want the new model to learn not just the engine, but how buyers actually refine it.

4. Train An Explanation Model

This model would not replace the stocking logic.

It would:

- explain recommendations in plain English
- summarize which rules fired
- call out risk factors
- translate model outputs for planners

This is a strong use case for an LLM, but it should sit on top of the deterministic or learned replenishment engine, not replace it.

Recommended Architecture

For your environment, I would recommend this order:

1. Keep the current rule engine as the source of truth.
2. Train a demand model.
3. Feed the demand model into the current rules.
4. Log both the raw demand forecast and the final rule-adjusted min/max.
5. Only after that, consider a mimic model or an explanation model.

In plain language:

- learn the uncertain part
- keep the policy part explicit

What Should Stay Hard-Coded

These rules should usually stay outside the learned model:

- ignored statuses
- location `9999` ignore rule
- `Regional` branch zero-max rule
- manual overrides
- supplier minimum amount floors
- branch-zero-max alerting
- DC-zero-max alerting
- low-cost local deviation threshold
- location ordering and DC designation

Why:

- these are business policy decisions
- they may change by management decision, not because demand changed
- they must remain auditable

What Is Safe To Learn

These parts are good candidates for machine learning:

- expected demand by location

- expected demand over lead time
- expected demand over the protection window
- seasonal ramp shape
- product-group trend influence
- whether branch demand is likely intermittent
- whether a recommendation will later be overridden by a buyer

Recommended Training Targets

Use different targets for different model types.

Demand-first model

Use:

- next month usage
- next 2 or 3 months usage
- demand over `lead_time + frequency`

Rule-mimic model

Use:

- `recommended_min_amount`
- `recommended_new_max`
- `recommended_min_delta`
- `recommendation_delta`

Also train separate classification heads for:

- `intermittent_branch_protection_applied`
- `seasonal_ramp_applied`
- `regional_zero_max_branch_pool_applied`
- `branch_zero_max_recommendation_alert`
- `dc_pooling_zero_max_alert`
- `low_cost_local_deviation_applied`
- `supplier_min_amount_floor_applied`
- `override_flag`

Ranking model

Use:

- probability the buyer accepts the recommendation
- expected size of the buyer adjustment
- risk of future override

The Main Training Tables To Build

You already have almost everything needed in the app.

From the raw workbook:

- supplier
- item
- location
- status
- ABC
- season
- MAC
- frequency
- lead time
- min
- max
- override fields
- `S. Min Amt`
- all 24 monthly usage columns

From the current engine:

- filtered means and std devs
- reference window decision
- seasonal ramp fields
- intermittent-demand fields
- pooled deviation fields
- policy floors
- final min/max targets
- alert flags

How To Extract Teacher Data From This Repo

The current repo already exposes the key pieces.

Load the workbook

Use `deviation_dash/data_loader.py`, especially:

- `load_data_workbook(...)`

This returns:

- normalized raw data
- detected `monthly_columns`

Run the current recommendation engine

Use `deviation_dash/recommendations.py`, especially:

- `METHOD_LOOKUP`
- `SeasonalityConfig`
- `calculate_method(...)`

Important detail:

- `calculate_method(...)` returns `location_detail` first and `item_summary` second

Example extraction code

```
from datetime import date
from pathlib import Path

from deviation_dash.acceleration import detect_acceleration
from deviation_dash.data_loader import load_data_workbook
from deviation_dash.recommendations import (
    DEFAULT_SERVICE_LEVELS,
    METHOD_LOOKUP,
    SeasonalityConfig,
    calculate_method,
)

workbook_path = Path("DataV3.xlsx")
loaded = load_data_workbook(
    workbook_path.read_bytes(),
    source_name=workbook_path.name,
)

location_detail, item_summary = calculate_method(
```

```
    loaded.data,
    loaded.monthly_columns,
    METHOD_LOOKUP["excl_0_devmean_24_round"],
    seasonality=SeasonalityConfig(
        enabled=True,
        planning_season="Summer",
        as_of_date=date(2026, 3, 23),
    ),
    acceleration=detect_acceleration(prefer_gpu=False),
    forecast_lookup=None,
    service_levels=dict(DEFAULT_SERVICE_LEVELS),
    cheap_local_deviation_threshold=1.0,
    remove_highest_outlier_enabled=False,
    highest_outlier_threshold_pct=200.0,
)
```

This gives you two very useful training tables:

- `location_detail` : one row per supplier-item-location recommendation
- `item_summary` : one row per supplier-item summary

Recommended Feature Sets

1. Raw demand history

Use:

- all 24 monthly columns
- rolling 3, 6, 12, and 24 month totals
- recent vs prior year totals
- count of non-zero months
- last non-zero month
- max month usage
- average of non-zero months

2. Intermittent-demand features

Use:

- active-month count
- ADI

- top-two-month concentration
- zero ratio
- largest month / median non-zero month
- branch vs DC flag

These are especially important for reproducing the intermittent branch protection logic.

3. Seasonality features

Use:

- month-of-year usage profile
- item season
- planning season
- current month
- distance to seasonal peak
- month-specific seasonal factors
- product-group seasonality

4. Policy features

Use:

- status
- whether status contains `Regional`
- ABC class
- service level percentage
- frequency
- frequency days
- lead time
- protection days
- MAC
- `S. Min Amt`
- current min
- current max
- override flag
- DC vs branch flag

5. Network features

Use:

- company total demand for the item
- branch share of company demand
- total deviation pooled to DC
- current DC max
- current DC min
- number of stocked branches
- number of zero-max branches

Best Labeling Strategy

Do not create only one label.

Build a layered teacher dataset:

Row-level labels

- final `recommended_min_amount`
- final `recommended_new_max`
- final alert flags

Intermediate labels

- `filtered_mean`
- `filtered_std_dev`
- `reference_window_months`
- `seasonal_ramp_applied`
- `intermittent_branch_protection_applied`
- `deviation_pooled_to_dc`
- `deviation_absorbed_by_dc`

Why this matters:

- intermediate labels make debugging much easier
- they let the team see where the learned model diverges
- they make it possible to train smaller models for specific sub-decisions

Best Modeling Patterns

Pattern A: Forecast + rules

Train:

- a forecasting model for demand

Keep explicit:

- all downstream policy logic

Best for:

- production safety
- explainability
- easier rollout

Pattern B: Multi-head student model

Train one model with multiple outputs:

- min regression head
- max regression head
- intermittent flag classifier
- seasonal ramp classifier
- alert classifiers

Best for:

- faster inference
- mimicking the teacher engine

Main caution:

- requires strong feature engineering
- easier to drift away from business rules if not constrained

Pattern C: Two-stage model

Stage 1:

- predict demand class or item behavior class

Stage 2:

- use specialized models for each class

Examples:

- intermittent items
- steady items
- strong seasonal items
- low-cost items
- regional items

This often works better than forcing one model to learn every behavior equally.

How To Split The Data

Use more than one evaluation split.

Time split

Train on older months and validate on newer months.

This is the most important split because replenishment is a forward-looking problem.

Item holdout split

Hold out entire items.

This shows whether the model can generalize to new item histories.

Supplier holdout split

Hold out some suppliers.

This tests whether the model depends too heavily on supplier-specific patterns.

Cold-start split

Create a special set for:

- few non-zero months
- zero-max branches
- regional items
- override items

These are the edge cases that usually break first.

How To Evaluate The New Models

Do not measure only one metric.

Use:

- MAE on `recommended_min_amount`
- MAE on `recommended_new_max`
- exact-match rate on alert flags
- precision/recall on intermittent-demand detection
- precision/recall on regional branch zero-max handling
- error on pooled-to-DC quantities
- sign accuracy on min/max deltas

Also track business-facing metrics:

- stockout rate
- fill rate
- turns
- excess inventory
- percent of buyer overrides after model recommendation

A Practical Training Workflow

Step 1. Freeze a teacher configuration

Choose one official teacher setup:

- active method
- service levels
- seasonality settings
- low-cost threshold
- highest-month normalization setting

Do not let these drift while the team is building its first training set.

Step 2. Generate teacher outputs for many snapshots

For each historical workbook snapshot:

- load raw data
- run the current engine
- save raw rows
- save location-level outputs
- save item-level summaries
- stamp the configuration version used

This gives you a reproducible supervised-learning dataset.

Step 3. Build a feature store

Store:

- raw workbook columns
- derived demand statistics
- rule intermediate values
- final outputs

Keep every row keyed by:

- supplier
- item
- location
- snapshot date

Step 4. Train a simple baseline first

Start with:

- XGBoost or LightGBM for min/max regression
- XGBoost or LightGBM classifiers for key flags

Do this before a deep model.

It will show whether the dataset is behaving logically.

Step 5. Add a demand model

Use:

- AutoGluon TimeSeries if the team wants a strong out-of-the-box forecaster

- or a custom model if they want tighter control

Then compare:

- teacher-only engine
- forecast + rules
- pure mimic model

Step 6. Compare against buyer behavior

If you have buyer edits and override history, compare:

- teacher output vs buyer final decision
- learned model vs buyer final decision

This is where the project becomes truly intelligent instead of just a rule clone.

What Not To Do

- Do not train only on final max with no rule context.
- Do not let the model learn overrides as if they are demand.
- Do not mix different teacher configurations without versioning them.
- Do not evaluate only average numeric error.
- Do not replace hard policy rules with a black box too early.

Recommended First Deliverables For Your Programmers

I would ask the team to build these in order:

1. A dataset generator that runs the current engine and exports teacher rows.
2. A baseline min/max mimic model.
3. A separate intermittent-demand classifier.
4. A demand forecast model.
5. A hybrid forecast + rules pipeline.
6. A comparison dashboard showing teacher vs student vs buyer decision.

Best End State

The best final design is likely:

- a learned demand model
- the current business-rule layer kept explicit
- a learned explanation layer on top
- buyer feedback logged and used for future retraining

That gives you:

- better forecasting
- safe policy control
- good explainability
- a path to continuous improvement

Short Version

If the team only remembers one thing, it should be this:

- use the current engine as a teacher
- learn demand, not policy
- keep hard rules explicit
- train on both intermediate logic and final outputs
- compare everything against real buyer decisions when possible